

Simulation framework for Table 1

J. Dedecker, M.-L. Taupin, V. Runge and A.-S. Tocquet

2026-07-01

Contents

1	Objective	1
2	Packages	1
3	Global parameters	1
4	Fast Toeplitz-Gaussian generator	2
5	One Table 1 data set	3
6	Shared metadata	4
7	Example data set	5
8	Screening function from the <code>mfscreen</code> package	6
9	One replication: data generation and screening	7
10	Example: one replication	8
11	Monte Carlo simulation for one Table 1 column	9
12	Generate Table 1	10
13	Generated Table 1	10
14	Selection frequencies for X1 to X18	11
15	FPR–SSP threshold curves: screening score versus Pearson correlation	12
15.1	One threshold-curve replication	13
15.2	Monte Carlo threshold curves	14
15.3	Mean Pearson correlations	16
15.4	FPR–SSP plot	16
15.5	FPR–SSP curve tables	18

1 Objective

This document implements the continuous, heteroscedastic Table 1 data-generating process with a faster simulator for the correlated Gaussian blocks.

The improvement replaces dense covariance-matrix construction and Cholesky factorization by the exact stationary Gaussian AR(1) recursion

$$X_j = \rho X_{j-1} + \sqrt{1 - \rho^2} \varepsilon_j, \quad \varepsilon_j \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2),$$

which gives

$$\text{Cov}(X_j, X_k) = \sigma^2 \rho^{|j-k|}.$$

Thus the simulated covariance is the same Toeplitz covariance as in Table 1, but the large 1982-variable noise block is generated much faster.

2 Packages

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE, cache.path = "table1_cache_threshold_curves/")
library(parallel)
```

3 Global parameters

```
# Table 1 dimensions
p <- 2000L
N <- 500L

#set.seed(20260701)
base_seed <- 20260701L
replication_seeds <- base_seed + seq_len(N)

n_values <- c(500L, 1000L, 2000L, 2500L)

# Variable groups
n_active <- 10L
n_proxy_and_noise <- 8L
n_toeplitz_noise <- p - n_active - n_proxy_and_noise # 1982

# Toeplitz correlations
rho_active <- 0.57
rho_toeplitz_noise <- 0.70

# Regression coefficients, including the intercept
```

```

theta_star <- c(
  -1,
  3.619350,
  -3.274923,
  2.963273,
  -2.681280,
  2, 4, 6, 3, 2, 4
)

stopifnot(length(theta_star) == n_active + 1L)
stopifnot(n_toeplitz_noise == 1982L)

```

4 Fast Toeplitz-Gaussian generator

```

simulate_toeplitz_gaussian_fast <- function(n, d, rho, sigma = 1)
{
  stopifnot(d >= 1L, abs(rho) < 1, sigma > 0)

  X <- matrix(0, nrow = n, ncol = d)

  # A stationary AR(1) process starts with marginal variance sigma^2.
  X[, 1L] <- rnorm(n, mean = 0, sd = sigma)

  if (d > 1L) {
    innovation_sd <- sigma * sqrt(1 - rho^2)

    # Draw all innovations at once to reduce random-number-generation overhead.
    innovations <- matrix(
      rnorm(n * (d - 1L), mean = 0, sd = innovation_sd),
      nrow = n,
      ncol = d - 1L
    )

    # Each row is a stationary Gaussian AR(1) sequence.
    for (j in 2:d) {
      X[, j] <- rho * X[, j - 1L] + innovations[, j - 1L]
    }
  }

  X
}

```

5 One Table 1 data set

This function returns only X and Y by default. It does not create column names or duplicate metadata at every replication, reducing memory allocation in Monte Carlo experiments.

```

simulate_table1_dataset_fast <- function(
  n,
  p = 2000,
  rho_active = 0.57,
  rho_toeplitz_noise = 0.70,
  theta_star = c(-1, 3.619350, -3.274923, 2.963273, -2.681280, 2, 4, 6, 3, 2, 4),
  add_names = FALSE
) {
  # -----
  # Group A: X1,...,X10
  # -----

  X1_to_X5 <- simulate_toeplitz_gaussian_fast(
    n = n,
    d = 5L,
    rho = rho_active,
    sigma = 1
  )

  X6 <- rbinom(n, size = 1L, prob = 0.35)
  X7 <- rchisq(n, df = 2)
  X8 <- X1_to_X5[, 1L] * rpois(n, lambda = 2)
  X9 <- X1_to_X5[, 2L] * rnorm(n, mean = 1, sd = 1)
  X10 <- rt(n, df = 14)

  X_active <- cbind(X1_to_X5, X6, X7, X8, X9, X10)

  # -----
  # Response Y
  # -----

  linear_predictor <- drop(theta_star[1L] + X_active %*% theta_star[-1L])
  epsilon <- X_active[, 5L] * (rchisq(n, df = 3) - 3)
  Y <- abs(linear_predictor)^0.8 + epsilon / 3

  # -----
  # Groups B and C: X11,...,X18
  # -----

  Z3 <- rnorm(n, mean = 0, sd = 0.40)
  Z4 <- rnorm(n, mean = 0, sd = 0.02)
  Z5 <- rnorm(n, mean = 0, sd = 0.35)
  Z6 <- rnorm(n, mean = 0, sd = 0.55)

  X_proxy_and_noise <- cbind(
    X_active[, 1L] + Z3, # X11
    X_active[, 3L] + Z4, # X12
    X_active[, 4L] + Z5, # X13
    X_active[, 5L] + Z6, # X14
    Z3, Z4, Z5, Z6      # X15,...,X18
  )

  # -----

```

```

# Group D: X19,...,X2000
# -----

X19_to_X2000 <- simulateToeplitzGaussianFast(
  n = n,
  d = p - 18L,
  rho = rhoToeplitzNoise,
  sigma = 1
)

X <- cbind(X_active, X_proxy_and_noise, X19_to_X2000)

stopifnot(ncol(X) == p)
stopifnot(length(Y) == n)

if (add_names) {
  colnames(X) <- paste0("X", seq_len(p))
}

list(X = X, Y = Y)
}

```

6 Shared metadata

Keep this object once, outside Monte Carlo replications.

```

table1_metadata <- list(
  p = p,
  N = N,
  n_values = n_values,
  theta_star = theta_star,
  rho_active = rho_active,
  rhoToeplitzNoise = rhoToeplitzNoise,
  active_indices = 1:10,
  proxy_indices = 11:14,
  null_noise_indices = 15:18,
  toeplitz_noise_indices = 19:p,
  table1_positive_indices = 1:14,
  table1_null_indices = 15:p
)

```

7 Example data set

```

example_dataset <- simulateTable1DatasetFast(
  n = 500L,
  add_names = FALSE
)

dim(example_dataset$X)

```

```
## [1] 500 2000
```

```
head(example_dataset$X[, 1:12])
```

```
##
## [1,] 0.5305587 1.2978132 0.4403768 1.21895205 0.7892700 0 2.1405060
## [2,] 0.8710185 1.1415464 1.7147343 -0.98353597 -1.3728382 1 1.3713822
## [3,] 0.7114681 -1.6923016 -1.1385901 -2.04392382 -0.7407533 1 3.8831025
## [4,] -0.6944923 -1.0399208 1.5968422 -0.01573471 1.3723424 0 0.1904702
## [5,] -0.5456918 -0.4866166 2.3313683 3.49743574 2.0362339 0 16.3699344
## [6,] 0.2062990 -0.4796644 0.2613564 -0.23765159 -1.5203732 0 0.5536665
##
## X8 X9 X10
## [1,] 2.1222349 1.6798438 0.2426068 -0.2660988 0.4202757
## [2,] 1.7420370 0.1447827 -1.3605665 0.6673742 1.7264371
## [3,] 2.1344043 -1.5493905 -0.6344212 0.8334287 -1.1378577
## [4,] -0.6944923 -1.3850598 -0.1443740 -0.7209014 1.6064725
## [5,] -0.5456918 0.1167272 -2.5709457 -0.9549427 2.3125235
## [6,] 0.6188969 -0.6929815 0.3638662 -0.3303201 0.2603571
```

```
head(example_dataset$Y)
```

```
## [1] 10.324388 10.094220 16.735585 1.745110 36.654601 3.009737
```

```
# Pearson correlations between every predictor X_j and Y.
pearson_correlations_example <- cor(example_dataset$X, example_dataset$Y)
pearson_correlations_first18 <- data.frame(
  variable = paste0("X", 1:18),
  pearson_correlation_with_Y = pearson_correlations_example[1:18]
)

knitr::kable(
  pearson_correlations_first18,
  digits = 4,
  caption = "Pearson correlations between X1,...,X18 and Y in one simulated data set"
)
```

Table 1: Pearson correlations between X1,...,X18 and Y in one simulated data set

variable	pearson_correlation_with_Y
X1	0.4012
X2	0.2599
X3	0.2404
X4	0.1548
X5	0.1506
X6	0.1397
X7	0.7662
X8	0.3786
X9	0.2570
X10	0.1067
X11	0.3590

variable	pearson_correlation_with_Y
X12	0.2407
X13	0.1411
X14	0.1459
X15	-0.0173
X16	0.0061
X17	-0.0111
X18	0.0237

```
# Empirical correlation of X1,...,X5, to inspect the rho = 0.57 block:
round(cor(example_dataset$X[, 1:5]), 3)
```

```
##
##  1.000 0.585 0.312 0.161 0.069
##  0.585 1.000 0.545 0.303 0.130
##  0.312 0.545 1.000 0.535 0.310
##  0.161 0.303 0.535 1.000 0.622
##  0.069 0.130 0.310 0.622 1.000
```

```
# Empirical correlation of X150,...,X155, to inspect the rho = 0.7 block:
round(cor(example_dataset$X[, 150:155]), 3)
```

```
##
##  1.000 0.679 0.476 0.362 0.238 0.195
##  0.679 1.000 0.682 0.464 0.358 0.254
##  0.476 0.682 1.000 0.679 0.493 0.365
##  0.362 0.464 0.679 1.000 0.721 0.495
##  0.238 0.358 0.493 0.721 1.000 0.680
##  0.195 0.254 0.365 0.495 0.680 1.000
```

8 Screening function from the mfscreen package

The screening routine is provided by the GitHub package `vrunge/mfscreen`. The following chunk installs remotes only when needed, installs `mfscreen` only when it is not already available, and then loads the package.

```
if (!requireNamespace("remotes", quietly = TRUE)) {
  install.packages("remotes")
}

if (!requireNamespace("mfscreen", quietly = TRUE)) {
  remotes::install_github("vrunge/mfscreen")
}

library(mfscreen)

# Confirm that the required function is available.
stopifnot(exists("screening_test_matrix", where = asNamespace("mfscreen")))
```

9 One replication: data generation and screening

This function generates one Table 1 data set, immediately applies the Rcpp screening function, and returns only the quantities needed for Table 1. The $n \times p$ design matrix is not retained after the function returns.

```
run_one_table1_replication <- function(n, q = 0.10) {
  dat <- simulate_table1_dataset_fast(
    n = n,
    p = p,
    rho_active = rho_active,
    rho_toeplitz_noise = rho_toeplitz_noise,
    theta_star = theta_star,
    add_names = FALSE
  )

  test_result <- mfscreen::screening_test_matrix(
    X = dat$X,
    y = as.numeric(dat$Y),
    q = q
  )

  selected <- as.logical(test_result$selected)

  # Table 1: X1,...,X14 are treated as positive; X15,...,X2000 are null.
  selected_first18 <- as.integer(selected[1:18])

  output <- list(
    selected_first18 = selected_first18,
    selected_positive = sum(selected[1:14]),
    selected_null = sum(selected[15:p]),
    selected_total = sum(selected),
    threshold = test_result$threshold
  )

  output
}
```

10 Example: one replication

```
one_result <- run_one_table1_replication(n = 500L, q = 0.10)

one_replication_summary <- data.frame(
  n = 500L,
  p = p,
  q = 0.10,
  threshold = one_result$threshold,
  selected_among_X1_to_X14 = one_result$selected_positive,
  selected_among_X15_to_X2000 = one_result$selected_null,
  selected_model_size = one_result$selected_total,
  TPR_contribution = one_result$selected_positive / 14,
  FPR_contribution = one_result$selected_null / (p - 14)
```



```
)
knitr::kable(
  one_replication_summary,
  digits = 4,
  caption = "One replication: quantities that are averaged for Table 1"
)
```

Table 2: One replication: quantities that are averaged for Table 1

n	p	q	threshold	selected_ among_X1_selected_X14	selected_ among_X15_selected_X2000	del_TPR	PPV_contribution	PPV_contribution
500	2000	0.1	0.3696	12	207	219	0.8571	0.1042

```
knitr::kable(
  data.frame(
    variable = paste0("X", 1:18),
    selected = one_result$selected_first18
  ),
  caption = "One replication: selection indicators for X1 to X18"
)
```

Table 3: One replication: selection indicators for X1 to X18

variable	selected
X1	1
X2	1
X3	1
X4	0
X5	1
X6	1
X7	1
X8	1
X9	1
X10	1
X11	1
X12	1
X13	0
X14	1
X15	0
X16	0
X17	1
X18	0

11 Monte Carlo simulation for one Table 1 column

The function below runs N replications for a fixed sample size and returns only compact outputs. It does not save any X or Y matrices.

```

run_table1_column <- function(n, N = N, q = 0.10,
                             mc.cores = 1L,
                             base_seed = 20260701L)
{
  replication_seeds <- base_seed + seq_len(N)

  one_draw <- function(i) {
    set.seed(replication_seeds[i])
    run_one_table1_replication(n = n, q = q)
  }

  if (.Platform$OS.type == "windows" || mc.cores <= 1L) {
    repetitions <- lapply(seq_len(N), one_draw)
  } else {
    repetitions <- parallel::mclapply(
      X = seq_len(N),
      FUN = one_draw,
      mc.cores = mc.cores,
      mc.set.seed = FALSE,
      mc.preschedule = FALSE
    )
  }

  selection_first18 <- do.call(
    rbind,
    lapply(repetitions, `[`, "selected_first18")
  )

  selected_positive <- vapply(
    repetitions,
    `[`,
    numeric(1),
    "selected_positive"
  )

  selected_null <- vapply(
    repetitions,
    `[`,
    numeric(1),
    "selected_null"
  )

  selected_total <- vapply(
    repetitions,
    `[`,
    numeric(1),
    "selected_total"
  )

  list(
    n = n,
    N = N,

```

```

    q = q,
    threshold = repetitions[[1]]$threshold,
    selection_frequency_first18 = colMeans(selection_first18),
    TPR = mean(selected_positive / 14),
    FPR = mean(selected_null / (p - 14)),
    mean_selected_model_size = mean(selected_total)
  )
}

```

12 Generate Table 1

This chunk is evaluated when the document is knitted. It runs the simulation for $n = 500, 1000, 2000, 2500$, each with $N = 500$ Monte Carlo replications, and then creates the generated Table 1 in the PDF.

For a quick development check, temporarily replace `N <- 500L` in the parameter section by a smaller value such as `N <- 5L`.

```

# On machines with limited RAM, start with 4 to 8 cores.
n_cores <- min(8L, parallel::detectCores(logical = FALSE))

table1_runs <- lapply(
  n_values,
  run_table1_column,
  N = N,
  q = 0.10,
  mc.cores = n_cores
)

names(table1_runs) <- paste0("n=", n_values)

```

13 Generated Table 1

The generated Table 1 reports one row for each sample size and the three Monte Carlo quantities reported in the paper: TPR, FPR, and the mean selected model size.

```

table1_summary <- data.frame(
  n = n_values,
  TPR = vapply(table1_runs, `[`, numeric(1), "TPR"),
  FPR = vapply(table1_runs, `[`, numeric(1), "FPR"),
  Mean_selected_model_size = vapply(
    table1_runs,
    `[`,
    numeric(1),
    "mean_selected_model_size"
  )
)

knitr::kable(
  table1_summary,
  format = "latex",
  booktabs = TRUE,

```

Table 4: Generated Table 1: Monte Carlo screening results ($N = 500$, $p = 2000$, $q = 0.10$)

	n	TPR	FPR	Mean_selected_model_size
n=500	500	0.816	0.103	216.722
n=1000	1000	0.899	0.102	214.478
n=2000	2000	0.961	0.101	213.930
n=2500	2500	0.977	0.101	214.250

```

digits = 3,
align = c("c", "c", "c", "c"),
caption = paste0(
  "Generated Table 1: Monte Carlo screening results ",
  "(N = ", N, ", p = ", p, ", q = 0.10)"
)
)

```

14 Selection frequencies for X1 to X18

This table contains the Monte Carlo selection frequencies for the first 18 covariates. The Table 1 TPR is the mean of the first 14 rows; the FPR is computed over all $X_{15}, \dots, X_{2000}$, not only $X_{15}, \dots, X_{18}$.

```

selection_first18_table <- data.frame(
  variable = paste0("X", 1:18),
  do.call(
    cbind,
    lapply(
      table1_runs,
      function(x) x$selection_frequency_first18
    )
  ),
  check.names = FALSE
)

colnames(selection_first18_table)[-1] <- names(table1_runs)

selection_first18_table[, -1] <- lapply(
  selection_first18_table[, -1, drop = FALSE],
  round,
  digits = 3
)

knitr::kable(
  selection_first18_table,
  format = "latex",
  booktabs = TRUE,
  caption = "Monte Carlo selection frequencies for X1 to X18"
)

```

Table 5: Monte Carlo selection frequencies for X1 to X18

variable	n=500	n=1000	n=2000	n=2500
X1	1.000	1.000	1.000	1.000
X2	0.998	1.000	1.000	1.000
X3	0.994	1.000	1.000	1.000
X4	0.372	0.524	0.766	0.856
X5	0.622	0.882	0.980	1.000
X6	0.598	0.860	0.988	0.996
X7	1.000	1.000	1.000	1.000
X8	0.994	1.000	1.000	1.000
X9	0.998	1.000	1.000	1.000
X10	0.990	1.000	1.000	1.000
X11	1.000	1.000	1.000	1.000
X12	0.992	1.000	1.000	1.000
X13	0.350	0.526	0.752	0.840
X14	0.518	0.800	0.968	0.990
X15	0.114	0.090	0.102	0.078
X16	0.108	0.086	0.114	0.118
X17	0.106	0.088	0.108	0.118
X18	0.104	0.108	0.114	0.118

15 FPR–SSP threshold curves: screening score versus Pearson correlation

For a given simulated data set, both procedures are evaluated over multiple threshold values.

- **Model-free screening:** the package is run for a grid of nominal false-positive levels `q_grid`. Each value gives one threshold and one selection set.
- **Pearson screening:** variables are retained when $|\text{cor}(X_j, Y)|$ exceeds a correlation threshold in `pearson_threshold_grid`.

Here, SSP denotes the **sure screening probability**: across Monte Carlo replications, the probability that every positive variable $X_1, \dots, X_{14}$ is retained. The FPR is the average proportion selected among the null variables $X_{15}, \dots, X_{2000}$.

```
# Use one n for the FPR--SSP curves; modify as needed.
n_curve <- 500L
N_curve <- N

# Several thresholds for model-free screening, parameterized by q.
q_grid <- c(0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.10, 0.20, 0.30, 0.50)

# Several absolute-Pearson-correlation cutoffs.
pearson_threshold_grid <- seq(0.01, 0.50, by = 0.01)

# Use an independent deterministic seed sequence for this experiment.
curve_base_seed <- 30260701L
```

15.1 One threshold-curve replication

This function computes all model-free selections and all Pearson selections from the same simulated X and Y . It also returns the Pearson correlation for each X_j - Y pair.

```
run_one_threshold_curve_replication <- function(
  n,
  q_grid,
  pearson_threshold_grid
) {
  dat <- simulate_table1_dataset_fast(
    n = n,
    p = p,
    rho_active = rho_active,
    rho_toeplitz_noise = rho_toeplitz_noise,
    theta_star = theta_star,
    add_names = FALSE
  )

  X <- dat$X
  Y <- dat$Y

  # Pearson correlation for every predictor X_j and Y.
  pearson_correlations <- as.numeric(cor(X, Y))

  # Model-free screening selections, one column per q value.
  screening_selected <- vapply(
    q_grid,
    function(q_value) {
      as.logical(
        mfscreen::screening_test_matrix(X = X, y = Y, q = q_value)$selected
      )
    },
    logical(ncol(X))
  )

  # Pearson selections, one column per correlation threshold.
  pearson_selected <- vapply(
    pearson_threshold_grid,
    function(cutoff) abs(pearson_correlations) >= cutoff,
    logical(ncol(X))
  )

  # vapply returns a vector when a grid has length one; force matrices.
  screening_selected <- matrix(
    screening_selected,
    nrow = ncol(X),
    ncol = length(q_grid)
  )
  pearson_selected <- matrix(
    pearson_selected,
    nrow = ncol(X),
    ncol = length(pearson_threshold_grid)
  )
}
```

```

list(
  screening_positive_count = colSums(screening_selected[1:14, , drop = FALSE]),
  screening_null_count = colSums(screening_selected[15:p, , drop = FALSE]),
  pearson_positive_count = colSums(pearson_selected[1:14, , drop = FALSE]),
  pearson_null_count = colSums(pearson_selected[15:p, , drop = FALSE]),
  pearson_correlations_first18 = pearson_correlations[1:18]
)
}

```

15.2 Monte Carlo threshold curves

```

run_threshold_curve_study <- function(
  n,
  N,
  q_grid,
  pearson_threshold_grid,
  base_seed,
  mc.cores = 1L
) {
  seeds <- base_seed + seq_len(N)

  one_draw <- function(i) {
    set.seed(seeds[i])
    run_one_threshold_curve_replication(
      n = n,
      q_grid = q_grid,
      pearson_threshold_grid = pearson_threshold_grid
    )
  }

  reps <- if (.Platform$OS.type == "windows" || mc.cores <= 1L) {
    lapply(seq_len(N), one_draw)
  } else {
    parallel::mclapply(
      X = seq_len(N),
      FUN = one_draw,
      mc.cores = mc.cores,
      mc.set.seed = FALSE,
      mc.preschedule = FALSE
    )
  }

  screening_positive <- do.call(
    rbind,
    lapply(reps, `[`, "screening_positive_count")
  )
  screening_null <- do.call(
    rbind,
    lapply(reps, `[`, "screening_null_count")
  )
  pearson_positive <- do.call(

```

```

    rbind,
    lapply(reps, `[`, "pearson_positive_count")
  )
  pearson_null <- do.call(
    rbind,
    lapply(reps, `[`, "pearson_null_count")
  )
  pearson_first18 <- do.call(
    rbind,
    lapply(reps, `[`, "pearson_correlations_first18")
  )

  # SSP = P(all 14 positive variables are retained).
  screening_curve <- data.frame(
    method = "Model-free screening",
    threshold_parameter = q_grid,
    threshold_label = paste0("q = ", q_grid),
    FPR = colMeans(screening_null / (p - 14)),
    SSP = colMeans(screening_positive == 14),
    mean_positive_selection_rate = colMeans(screening_positive / 14)
  )

  pearson_curve <- data.frame(
    method = "Pearson correlation",
    threshold_parameter = pearson_threshold_grid,
    threshold_label = paste0("|r| >= ", pearson_threshold_grid),
    FPR = colMeans(pearson_null / (p - 14)),
    SSP = colMeans(pearson_positive == 14),
    mean_positive_selection_rate = colMeans(pearson_positive / 14)
  )

  list(
    curve = rbind(screening_curve, pearson_curve),
    mean_pearson_correlations_first18 = colMeans(pearson_first18),
    sd_pearson_correlations_first18 = apply(pearson_first18, 2, sd)
  )
}

n_cores_curve <- min(8L, parallel::detectCores(logical = FALSE))

threshold_curve_results <- run_threshold_curve_study(
  n = n_curve,
  N = N_curve,
  q_grid = q_grid,
  pearson_threshold_grid = pearson_threshold_grid,
  base_seed = curve_base_seed,
  mc.cores = n_cores_curve
)

threshold_curve <- threshold_curve_results$curve

```


Table 6: Pearson correlations with Y across 500 simulations (n = 500)

variable	mean_pearson_correlation_with_Y	sd_pearson_correlation_with_Y
X1	0.3701	0.0425
X2	0.2053	0.0447
X3	0.1775	0.0428
X4	0.0565	0.0456
X5	0.0890	0.0452
X6	0.0861	0.0450
X7	0.6882	0.0351
X8	0.3528	0.0551
X9	0.1953	0.0446
X10	0.1787	0.0434
X11	0.3428	0.0433
X12	0.1774	0.0428
X13	0.0534	0.0462
X14	0.0805	0.0462
X15	-0.0028	0.0444
X16	-0.0011	0.0431
X17	0.0004	0.0452
X18	0.0051	0.0474

15.3 Mean Pearson correlations

These are the mean Pearson correlations across the `N_curve` simulated data sets, with their Monte Carlo standard deviations.

```
pearson_correlation_summary <- data.frame(
  variable = paste0("X", 1:18),
  mean_pearson_correlation_with_Y =
    threshold_curve_results$mean_pearson_correlations_first18,
  sd_pearson_correlation_with_Y =
    threshold_curve_results$sd_pearson_correlations_first18
)

knitr::kable(
  pearson_correlation_summary,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  caption = paste0(
    "Pearson correlations with Y across ", N_curve,
    " simulations (n = ", n_curve, ")"
  )
)
```

15.4 FPR-SSP plot

Each point corresponds to one threshold value in the relevant grid. Higher SSP is better; lower FPR is better.

```

screening_curve <- subset(threshold_curve, method == "Model-free screening")
pearson_curve <- subset(threshold_curve, method == "Pearson correlation")

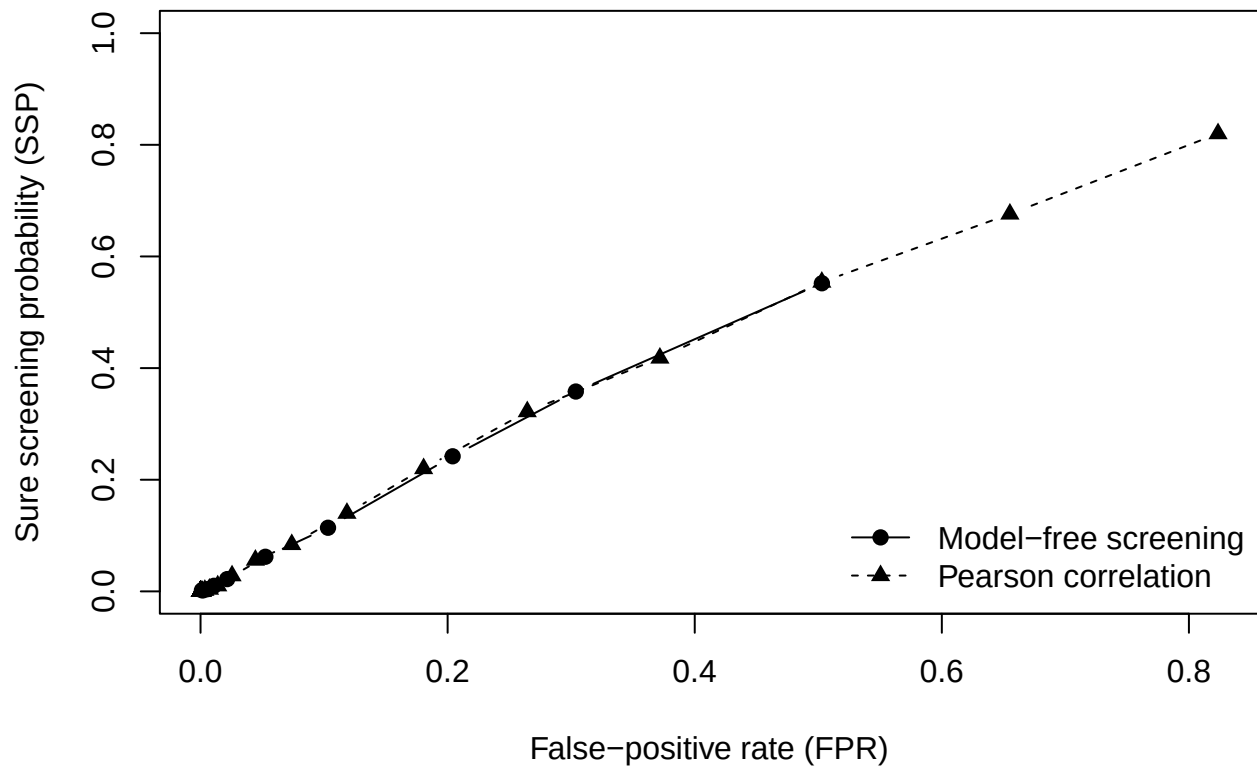
plot(
  screening_curve$FPR,
  screening_curve$SSP,
  type = "b",
  pch = 19,
  xlim = range(threshold_curve$FPR),
  ylim = c(0, 1),
  xlab = "False-positive rate (FPR)",
  ylab = "Sure screening probability (SSP)",
  main = paste0("FPR--SSP curves (n = ", n_curve, ", N = ", N_curve, ")")
)

lines(
  pearson_curve$FPR,
  pearson_curve$SSP,
  type = "b",
  pch = 17,
  lty = 2
)

legend(
  "bottomright",
  legend = c("Model-free screening", "Pearson correlation"),
  pch = c(19, 17),
  lty = c(1, 2),
  bty = "n"
)

```

FPR--SSP curves (n = 500, N = 500)



15.5 FPR--SSP curve tables

```
knitr::kable(
  screening_curve,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  caption = "FPR--SSP points for model-free screening"
)
```

```
knitr::kable(
  pearson_curve,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  caption = "FPR--SSP points for Pearson-correlation screening"
)
```

16 Save compact results

Table 7: FPR–SSP points for model-free screening

method	threshold_parameter	threshold_label	FPR	SSP	mean_positive_selection_rate
Model-free screening	0.001	q = 0.001	0.0011	0.002	0.5977
Model-free screening	0.002	q = 0.002	0.0022	0.002	0.6213
Model-free screening	0.005	q = 0.005	0.0056	0.004	0.6617
Model-free screening	0.010	q = 0.01	0.0109	0.010	0.6923
Model-free screening	0.020	q = 0.02	0.0216	0.022	0.7256
Model-free screening	0.050	q = 0.05	0.0525	0.062	0.7726
Model-free screening	0.100	q = 0.1	0.1032	0.114	0.8169
Model-free screening	0.200	q = 0.2	0.2041	0.242	0.8681
Model-free screening	0.300	q = 0.3	0.3038	0.358	0.8973
Model-free screening	0.500	q = 0.5	0.5029	0.552	0.9389

```

saveRDS(
  object = list(
    metadata = table1_metadata,
    table1_summary = table1_summary,
    table1_runs = table1_runs,
    threshold_curve = threshold_curve,
    pearson_correlation_summary = pearson_correlation_summary
  ),
  file = "table1_rcpp_screening_results.rds"
)

write.csv(
  table1_summary,
  file = "table1_rcpp_screening_summary.csv",
  row.names = FALSE
)

```

Table 8: FPR-SSP points for Pearson-correlation screening

	method	threshold_parameter	threshold_label	FPR	SSP	mean_positive_selection_rate
11	Pearson correlation	0.01	$ r \geq 0.01$	0.8236	0.820	0.9819
12	Pearson correlation	0.02	$ r \geq 0.02$	0.6551	0.676	0.9630
13	Pearson correlation	0.03	$ r \geq 0.03$	0.5029	0.554	0.9387
14	Pearson correlation	0.04	$ r \geq 0.04$	0.3718	0.418	0.9126
15	Pearson correlation	0.05	$ r \geq 0.05$	0.2645	0.322	0.8889
16	Pearson correlation	0.06	$ r \geq 0.06$	0.1806	0.220	0.8601
17	Pearson correlation	0.07	$ r \geq 0.07$	0.1185	0.140	0.8281
18	Pearson correlation	0.08	$ r \geq 0.08$	0.0738	0.084	0.7973
19	Pearson correlation	0.09	$ r \geq 0.09$	0.0444	0.056	0.7651
20	Pearson correlation	0.10	$ r \geq 0.1$	0.0255	0.028	0.7383
21	Pearson correlation	0.11	$ r \geq 0.11$	0.0139	0.010	0.7084
22	Pearson correlation	0.12	$ r \geq 0.12$	0.0072	0.004	0.6786
23	Pearson correlation	0.13	$ r \geq 0.13$	0.0035	0.002	0.6483
24	Pearson correlation	0.14	$ r \geq 0.14$	0.0016	0.000	0.6207
25	Pearson correlation	0.15	$ r \geq 0.15$	0.0007	0.000	0.5876
26	Pearson correlation	0.16	$ r \geq 0.16$	0.0003	0.000	0.5541
27	Pearson correlation	0.17	$ r \geq 0.17$	0.0001	0.000	0.5207
28	Pearson correlation	0.18	$ r \geq 0.18$	0.0000	0.000	0.4883
29	Pearson correlation	0.19	$ r \geq 0.19$	0.0000	0.000	0.4581
30	Pearson correlation	0.20	$ r \geq 0.2$	0.0000	0.000	0.4244
31	Pearson correlation	0.21	$ r \geq 0.21$	0.0000	0.000	0.3926
32	Pearson correlation	0.22	$ r \geq 0.22$	0.0000	0.000	0.3676
33	Pearson correlation	0.23	$ r \geq 0.23$	0.0000	0.000	0.3476
34	Pearson correlation	0.24	$ r \geq 0.24$	0.0000	0.000	0.3264
35	Pearson correlation	0.25	$ r \geq 0.25$	0.0000	0.000	0.3086
36	Pearson correlation	0.26	$ r \geq 0.26$	0.0000	0.000	0.2979
37	Pearson correlation	0.27	$ r \geq 0.27$	0.0000	0.000	0.2869
38	Pearson correlation	0.28	$ r \geq 0.28$	0.0000	0.000	0.2791
39	Pearson correlation	0.29	$ r \geq 0.29$	0.0000	0.000	0.2704
40	Pearson correlation	0.30	$ r \geq 0.3$	0.0000	0.000	0.2607
41	Pearson correlation	0.31	$ r \geq 0.31$	0.0000	0.000	0.2481
42	Pearson correlation	0.32	$ r \geq 0.32$	0.0000	0.000	0.2359
43	Pearson correlation	0.33	$ r \geq 0.33$	0.0000	0.000	0.2234
44	Pearson correlation	0.34	$ r \geq 0.34$	0.0000	0.000	0.2079
45	Pearson correlation	0.35	$ r \geq 0.35$	0.0000	0.000	0.1933
46	Pearson correlation	0.36	$ r \geq 0.36$	0.0000	0.000	0.1757
47	Pearson correlation	0.37	$ r \geq 0.37$	0.0000	0.000	0.1567
48	Pearson correlation	0.38	$ r \geq 0.38$	0.0000	0.000	0.1391
49	Pearson correlation	0.39	$ r \geq 0.39$	0.0000	0.000	0.1203
50	Pearson correlation	0.40	$ r \geq 0.4$	0.0000	0.000	0.1079
51	Pearson correlation	0.41	$ r \geq 0.41$	0.0000	0.000	0.0979
52	Pearson correlation	0.42	$ r \geq 0.42$	0.0000	0.000	0.0889
53	Pearson correlation	0.43	$ r \geq 0.43$	0.0000	0.000	0.0834
54	Pearson correlation	0.44	$ r \geq 0.44$	0.0000	0.000	0.0786
55	Pearson correlation	0.45	$ r \geq 0.45$	0.0000	0.000	0.0761
56	Pearson correlation	0.46	$ r \geq 0.46$	0.0000	0.000	0.0749
57	Pearson correlation	0.47	$ r \geq 0.47$	0.0000	0.000	0.0730
58	Pearson correlation	0.48	$ r \geq 0.48$	0.0000	0.000	0.0721
59	Pearson correlation	0.49	$ r \geq 0.49$	0.0000	0.000	0.0719
60	Pearson correlation	0.50	$ r \geq 0.5$	0.0000	0.000	0.0716